

Helix Thready — Container Topology

Table of Contents

- 1. Design constraints
- 2. Per-environment topology diagram
- 3. Service inventory
- 4. Container images (pinned)
- 5. Networks
- 6. Volumes & persistence
- 7. Resource limits
- 8. BUILD-NEW placeholders (no bluff)
- 9. HelixLLM as an external endpoint
- 10. Modelling the stack with the containers API
- 11. Open items

Rev	Date	Author	Change
1	2026-07-21	swarm (deployment)	Initial per-environment service inventory, images, networks, volumes, limits

This document is the authoritative inventory of **what containers make up one Helix Thready environment**, the images they run, how they are networked and volumed, their resource limits, and which of them are **BUILD-NEW placeholders** that do not yet run real code. The same stack is instantiated three times (dev / sta / prod), each in its own Podman Compose project and its own **host-port band**.

Diagram source: sibling under `diagrams/`. Rendered PNG/SVG exported via Docs Chain (\$11.4.65).

Table of Contents

- 1. Design constraints
- 2. Per-environment topology diagram
- 3. Service inventory
- 4. Container images (pinned)
- 5. Networks
- 6. Volumes & persistence
- 7. Resource limits

8. BUILD-NEW placeholders (no bluff)
9. HelixLLM as an external endpoint
10. Modelling the stack with the containers API
11. Open items

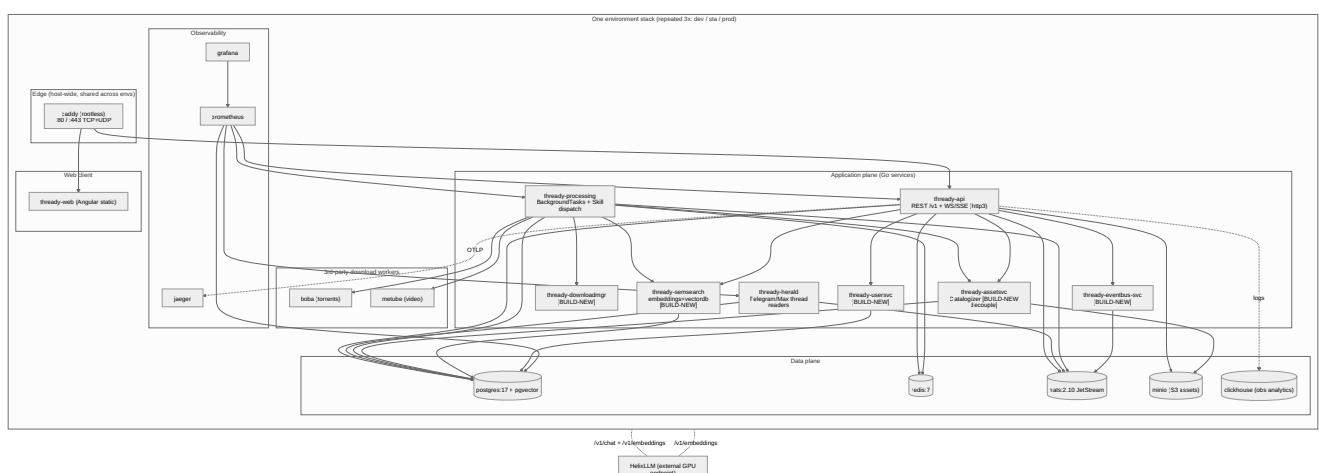
1. Design constraints

- **Rootless Podman only** — every container runs under the unprivileged `thready` user; no Docker daemon, no root, no `sudo` `[CONSTITUTION §11.4.76/161]`.
- **Full environment separation** — no container, network, volume or port is shared between `dev`, `staging` and `prod` (`§8.2`). The only host-wide shared component is the single edge reverse proxy that fans out by subdomain (see `environments.md`).
- **In-house first** — the data/infra containers are the standard images the `containers` module's `DefaultHelixServices()` already uses (Postgres, Redis, NATS, Prometheus, Grafana, Jaeger), plus MinIO and ClickHouse for Thready's asset and analytics needs `[IN-HOUSE: containers]`.
- **Large-scale posture** — assets on a MinIO/S3 object tier (50 TB+), Postgres partitioned for 10k+ posts/day, JetStream as the primary event transport `[OPERATOR: scale]`.

Anti-bluff note. `basic-digital/containers` ships a `DefaultHelixServices()` helper that hardcodes **dev-only** credentials (`POSTGRES_PASSWORD=helix` , `GF_SECURITY_ADMIN_PASSWORD=admin` , a Vault dev root token) as *documented fallback defaults for local bring-up only*. Thready **does not** use that 20-service dev stack verbatim; it constructs its own `[]HelixService` and injects every credential from the environment (see [secrets-and-config.md](#)).

[GAP: #12 anti-bluff]

2. Per-environment topology diagram



Explanation (for readers/models that cannot see the diagram). The box labelled *EnvStack* is one complete environment; it is created three times, once per environment, in disjoint port bands. The single host-wide **edge** (Caddy) is drawn inside the box for context but is actually shared — it is the only component bound to the public 80/443 and it forwards to each environment's `thready-api` and static `thready-web` by subdomain.

The **application plane** is Thready's own Go services. `thready-api` is the public surface: the REST `/v1` API plus the WebSocket/SSE hub, served over HTTP/3 via `vasic-digital/http3`. `thready-herald` is the extended Herald that reads Telegram (via `gotd/td` MTPROTO) and Max threads and writes raw posts to Postgres while emitting `post.received` onto NATS JetStream. `thready-processing` is the engine: it consumes those events through a Postgres-backed BackgroundTasks queue (idempotent single-claim per post), classifies by hashtag/content-type, and dispatches the matching Skill(s) — calling out to `thready-assetsvc` (Catalogizer-based asset store), `thready-downloadmgr` (generic downloads), `thready-semsearch` (embeddings + pgvector), and the 3rd-party workers `boba` (torrents) and `metube` (video). `thready-usersvc` (three-tier RBAC), `thready-eventbus-svc` (client-facing subscription), and `thready-semsearch` back the API.

The **data plane** holds the stateful containers: PostgreSQL with the pgvector extension (both the system-of-record *and* the vector store live in one instance, per §2.1.1), Redis for the cache tier, NATS with JetStream for durable events, MinIO for the 50 TB+ asset object store, and ClickHouse for observability analytics. The **observability** trio (Prometheus scraping every Go service's `/metrics`, Grafana for dashboards, Jaeger for OTLP traces) is deployed per environment so a dev incident can never perturb prod telemetry. Finally, because the GPU lives on the operator workstation, **HelixLLM sits outside the compose file** and is reached as an external endpoint over its OpenAI-compatible `/v1/chat/completions` and `/v1/embeddings` routes — this is why the arrows to `LLM` are dotted and cross the environment boundary.

3. Service inventory

Legend — **Kind:** `data` (stateful infra) · `app` (Thready Go service) · `worker` (3rd-party) · `obs` (observability) · `edge` (host-wide). **Host-exposed:** whether a host port is published (all host ports are bound to `127.0.0.1` inside the env's band, except the edge). **Status** per the gap register.

Service	Kind	Internal port	Host-exposed	Status	Provenance
<code>caddy</code> (edge)	edge	80, 443 (TCP+UDP)	yes, <code>0.0.0.0</code>	PRODUCTION	[IN-HOUSE: <code>lets_encrypt</code> <code>reload target</code>]
<code>thready-postgres</code>	data	5432	loopback	PRODUCTION	[IN-HOUSE: <code>database</code>]
<code>thready-redis</code>	data	6379	loopback	PRODUCTION	[IN-HOUSE: <code>cache</code>]
<code>thready-nats</code>	data	4222 (mon 8222)	loopback	PRODUCTION	[IN-HOUSE: <code>eventbus</code>]
<code>thready-minio</code>	data	9000 (console 9001)	loopback	PRODUCTION	[IN-HOUSE: <code>storage</code>]
<code>thready-clickhouse</code>	data	8123 (native 9009)	loopback	PRODUCTION	[IN-HOUSE: <code>observability</code>]
<code>thready-api</code>	app	8443	loopback	FOUNDATION	[IN-HOUSE: <code>http3</code> , <code>auth</code> , <code>middleware</code>]
<code>thready-web</code>	app	8088	loopback	FOUNDATION	[IN-HOUSE: <code>design_system</code>]
<code>thready-herald</code>	app	7080	loopback	FOUNDATION (Telegram) / P0 gap (Max)	[GAP: #3 <code>herald</code>]
<code>thready-processing</code>	app	— (internal)	no	P0 gap (skill dispatch engine)	[GAP: #6 <code>skills</code>]
<code>thready-assetsvc</code>	app	8081	loopback	BUILD-NEW (decouple)	[GAP: #9/#20]
<code>thready-downloadmgr</code>	app	8082	loopback	BUILD-NEW	[GAP: #4/#20]
<code>thready-usersvc</code>	app	8083	loopback	BUILD-NEW	[GAP: #20]
<code>thready-eventbus-svc</code>	app	8084	loopback	BUILD-NEW	[GAP: #20]
<code>thready-semsearch</code>	app	8085	loopback	BUILD-NEW	[GAP: #20 + #1 <code>embedder</code>]

Service	Kind	Internal port	Host-exposed	Status	Provenance
boba	worker	8000	loopback	FOUNDATION	[IN-HOUSE: Boba]
metube	worker	8091	loopback	FOUNDATION / P0 gap (webhook)	[GAP: #5 metube]
thready-prometheus	obs	9090	loopback	PRODUCTION	[IN-HOUSE: observability]
thready-grafana	obs	3000	loopback	PRODUCTION	[IN-HOUSE: observability]
thready-jaeger	obs	16686 (OTLP 4317)	loopback	PRODUCTION	[IN-HOUSE: observability]

Why loopback host-ports? Inter-container traffic uses the compose network and DNS names (`thready-postgres:5432`), so almost nothing *needs* a host port. Host ports are published on `127.0.0.1` only for (a) the edge→api hop, (b) operator debugging over an SSH tunnel, and (c) Prometheus scraping — never on `0.0.0.0` except the edge. This keeps the attack surface to ports 22/80/443 (see [hetzner-provisioning.md](#) firewall).

4. Container images (pinned)

Images are pinned by digest in the real compose files; tags shown here for readability. Thready application images are built locally from each submodule's `Containerfile` and tagged `THREADY-<component>-<version>` (project-prefixed tags, `$11.4.151`).

Service	Image (tag)	Source
<code>caddy</code>	<code>docker.io/library/caddy:2.8-alpine</code>	upstream (edge only; ACME disabled — certs come from <code>lets_encrypt</code>)
<code>thready-postgres</code>	<code>docker.io/library/postgres:17-alpine</code> + <code>pgvector</code>	upstream + extension
<code>thready-redis</code>	<code>docker.io/library/redis:7-alpine</code>	upstream
<code>thready-nats</code>	<code>docker.io/library/nats:2.10-alpine</code>	upstream (JetStream enabled)
<code>thready-minio</code>	<code>docker.io/minio/minio:latest</code> (pin digest)	upstream
<code>thready-clickhouse</code>	<code>docker.io/clickhouse/clickhouse-server:24-alpine</code>	upstream
<code>thready-api</code> etc.	<code>localhost/thready-<component>:THREADY-<ver></code>	built from the submodule <code>Containerfile</code>
<code>boba</code>	<code>localhost/boba:<ver></code>	<code>milos85vasic/Boba-Base</code>
<code>metube</code>	<code>localhost/metube:<ver></code>	<code>milos85vasic/YT-DLP</code>
<code>thready-prometheus</code>	<code>docker.io/prom/prometheus:v2.50.0</code>	upstream (matches <code>containers</code> default)
<code>thready-grafana</code>	<code>docker.io/grafana/grafana:10.4.0</code>	upstream
<code>thready-jaeger</code>	<code>docker.io/jaegertracing/all-in-one:1.55</code>	upstream (<code>COLLECTOR_OTLP_ENABLED=true</code>)

VERIFIED: the Prometheus/Grafana/Jaeger/Postgres/Redis/NATS image choices mirror the pinned images in `vasic-digital/containers` `pkg/compose/helix_project.go::DefaultHelixServices()` (read at source). MinIO and ClickHouse are Thready additions for the asset tier and log analytics.

5. Networks

Each environment declares **one internal bridge network**, isolated per Compose project:

```
# fragment of compose.thready.<env>.yaml
networks:
  thready-<env>-net:
    driver: bridge          # rootless: netavark/aardvark-dns provides in-network DNS
    internal:
      false                # egress allowed (ACME, image pulls, messenger APIs); ingress only via edge
```

- **In-network DNS** resolves service names (`thready-postgres`) automatically under rootless Podman via `aardvark-dns` .
- Networks are **not** shared across environments: `thready-dev-net` , `thready-sta-net` , `thready-prod-net` are three separate bridges. A container in `dev` cannot reach a container in `prod` even by IP.
- The edge proxy joins **all three** networks (plus its own) so it can forward to each env's API, or — the default — reaches them via the loopback host-ports in each band, which avoids cross-joining and keeps the edge decoupled. `[DEFAULT — adjustable]`

6. Volumes & persistence

Named volumes per environment (Podman rootless volumes live under `/home/thready/.local/share/containers/storage/volumes/`):

Volume	Mounted by	Contents	Backup class
<code>thready-<env>-pgdata</code>	<code>thready-postgres</code>	Postgres cluster incl. pgvector	daily full + hourly WAL
<code>thready-<env>-pgwal</code>	<code>thready-postgres</code>	archived WAL segments (PITR)	continuous → secondary
<code>thready-<env>-minio</code>	<code>thready-minio</code>	asset objects (raw + <code>...-web</code> renditions)	daily snapshot/dedup
<code>thready-<env>-redis</code>	<code>thready-redis</code>	cache (ephemeral, RDB optional)	none (rebuildable)
<code>thready-<env>-nats</code>	<code>thready-nats</code>	JetStream streams/ consumers	daily
<code>thready-<env>-clickhouse</code>	<code>thready-clickhouse</code>	logs/metrics analytics	daily (or drop; rebuildable)
<code>thready-<env>-grafana</code>	<code>thready-grafana</code>	dashboards/config	daily
<code>certs</code> (bind)	<code>caddy</code>	live TLS certs from <code>lets_encrypt</code>	included in config backup

Backup classes are defined in `backup-dr.md`. Postgres WAL archiving to `thready-<env>-pgwal` (and onward to the secondary store) is what delivers the RPO $\approx$ 1 h target.

[GAP: #3.2] **MinIO signed-URL parity.** The gap register flags that `storage`’s CloudFront-style signing is AWS-specific; Thready runs self-hosted MinIO, so the deploy checklist includes a **signed-URL round-trip test against the deployed MinIO** before assets go live (tracked in [backup-dr.md §6](#)).

7. Resource limits

Resource limits use the `containers HelixResourceLimits` shape (`CPUs` , `Memory` , `Pids`), which maps to Podman `--cpus` / `--memory` / `--pids-limit` . Production baseline `[DEFAULT – adjustable]` :

Service	CPUs	Memory	Notes
<code>thready-postgres</code> (prod)	6.0	24 GiB	NVMe-backed; <code>shared_buffers</code> $\approx$ 25 % RAM; pgvector HNSW cache
<code>thready-minio</code> (prod)	2.0	4 GiB	I/O bound; disk is the constraint
<code>thready-processing</code> (prod)	4.0	8 GiB	32 BackgroundTasks workers (Q4)
<code>thready-api</code> (prod)	2.0	2 GiB	stateless; scale out if p95 > 150 ms
<code>thready-clickhouse</code>	2.0	4 GiB	analytics only
dev / sta each service	$\approx$ 1/3 of prod	$\approx$ 1/3 of prod	smaller footprint; three stacks co-reside

Because all three stacks co-reside on one host, the sum of prod + sta + dev limits must fit the host (see [hetzner-provisioning.md §2](#) for the ≥ 16 vCPU / ≥ 64 GB baseline). `dev` / `sta` are intentionally throttled so `prod` keeps headroom.

8. BUILD-NEW placeholders (no bluff)

The following containers are **declared in the compose files but their images are placeholders** — they do not yet run production code. They correspond to confirmed `[BUILD-NEW]` gaps (`§4.2.1` , gap-register §11). Until built, the compose file marks them with `profiles: [buildnew]` so a default `up` **does not** start a non-working container and pretend the stack is complete.

Placeholder service	Gap	Blocking?	Plan to close
<code>thready-downloadmgr</code>	[GAP: #4 filesystem/DownloadManager, P0]	Yes (media pipeline)	New Go submodule: HTTP/2+3 source + queue/resume/segment/progress/callback reusing <code>filesystem</code>
<code>thready-usersvc</code>	[GAP: #20, P0]	Yes (multi-tenant auth)	Build on <code>auth</code> + <code>security/pkg/policy</code> + Catalogizer RBAC; RS256/EdDSA + TOTP
<code>thready-semsearch</code>	[GAP: #20 + #1, P0]	Yes (search SLO)	<code>embeddings</code> + <code>vectordb</code> (pgvector) + <code>MCP_Module</code> over HelixLLM; must set <code>HELIX_EMBEDDING_PROVIDER=lama</code>
<code>thready-assetsvc</code>	[GAP: #9 decouple, P1]	Partial	Decouple Asset Service from Catalogizer; add HLS/DASH + <code>...-web</code> renditions
<code>thready-eventbus-svc</code>	[GAP: #20, P1]	No (API can read JetStream directly first)	Thin service wrapping <code>eventbus</code> /JetStream for client subscription
<code>thready-herald</code> (Max path)	[GAP: #3 herald, P0]	Yes (Max ingest)	Promote MTPROTO reader to first-class channel; build Max Bot API + OneMe WebSocket Go client
<code>metube</code> (webhook)	[GAP: #5 metube, P0]	Yes (async completion)	Add outbound completion webhook (shared callback schema)

This is the “no bluff” contract in practice: the deployment never green-lights a stack whose `buildnew` profile services are empty shells. The **deploy verification** asserts each *enabled* service passes a **real** health check, not a stub that always returns 200.

9. HelixLLM as an external endpoint

HelixLLM is not in the per-environment compose file because the GPU lives on the operator workstation (Q5). It is registered as a `ServiceEndpoint` (from `containers/pkg/endpoint`) that the processing/semsearch services resolve at runtime:

```
// thready processing/semsearch config → containers ServiceEndpoint
llm := endpoint.ServiceEndpoint{
    Host:      "http://gpu-node.helix.lan", // or the workstation tunnel
    Port:      "8080",
    URL:       "http://gpu-node.helix.lan:8080", // OpenAI-compatible base
    Enabled:    true,
    Required:   true,          // semsearch cannot work without it
    Remote:     true,          // NOT managed by the local runtime
    HealthType: "http",
    HealthPath: "/health",
    Timeout:    10 * time.Second,
    RetryCount: 3,
    DiscoveryEnabled: true,
    DiscoveryMethod: "dns",    // or mDNS on the LAN
}
```

`Remote: true` tells the `BootManager` this endpoint is *discovered/health-checked* but never `compose up` 'd locally. If HelixLLM is unreachable at boot, the semantic-search service (which is `Required`) fails the boot gate loudly rather than silently degrading — consistent with the `[GAP: #1]` rule that a missing real embedder must **fail, not warn**.

10. Modelling the stack with the containers API

Thready's stack is expressed as a `[]compose.HelixService` and driven through the `containers` orchestrator. The types below are **verified** from `vasic-digital/containers/pkg/compose/helix_project.go`:

```

import (
    "time"
    "digital.vasic.containers/pkg/compose"
)

// One environment's Thready data-plane fragment (credentials injected from env — never
// literals).
func threadyDataPlane(env string) []compose.HelixService {
    return []compose.HelixService{
        {
            Name: "thready-postgres",
            Image: "docker.io/library/postgres:17-alpine",
            Ports: []compose.PortMapping{{HostPort: 0, ContainerPort: 5432, Protocol:
                "tcp"}}, // HostPort resolved by port_prefix
            Env: map[string]string{
                "POSTGRES_USER": mustEnv("THREADY_" + env + "_PG_USER"),
                "POSTGRES_PASSWORD": mustEnv("THREADY_" + env + "_PG_PASSWORD"),
                "POSTGRES_DB": "thready",
            },
            Volumes: []string{"thready-" + env + "-pgdata:/var/lib/postgresql/data"},
            HealthCheck: &compose.HelixHealthCheck{
                Test: []string{"CMD-SHELL", "pg_isready -U $POSTGRES_USER -d thready"},
                Interval: 5 * time.Second, Timeout: 5 * time.Second, Retries: 5,
            },
            ResourceLimits: &compose.HelixResourceLimits{CPUs: "6.0", Memory: "24g", Pids: 512},
        },
        // ... redis, nats, minio, clickhouse ...
    }
}

```

- `HelixService`, `PortMapping`, `HelixHealthCheck`, `HelixResourceLimits` are the exact structs the module exposes.
- `NewHelixComposeProject("thready-"+env, services)` builds the project; `podman-compose.md` shows how it is rendered and brought up rootlessly.
- `mustEnv` (a Thready helper, not a module function) fails fast if a secret env var is unset — the no-hardcoding rule `[CONSTITUTION §11.4.10]`.

11. Open items

- `[OPEN: buildnew-images]` — the placeholder services in §8 have no real images yet; the compose `buildnew` profile keeps them out of a default `up`. Tracked as ATM workable items in `development`.
- `[OPEN: pgvector-image]` — whether to use `pgvector/pgvector:pg17` directly or add the extension to the base `postgres:17-alpine` at build time is a build decision for the database area; both satisfy this topology.
- `[GAP: #3.2]` MinIO signed-URL parity test — tracked in `backup-dr.md`.

Made with love ♥ by Helix Development.